

Team Psiren: Jacob Bergeron, Skyler Dooley, Michael Monda, and Jacob Newman

Purpose

The purpose of this document is to provide evidence for the different roles of the team members.

Introduction

We developed an 'OpenTable' application mimic. OpenTable is a software that allows users to search restaurants and make reservations at these restaurants for their desired date, time, table, and number of guests. There are three types of users for our software: consumers, managers, and administrators.

When you first open the application, you are brought to the homepage where you have the following buttons: browse restaurants where the consumer can view all activated restaurants and also search for available reservations times by restaurant, search restaurants where the consumer can search for a specific restaurant that has open reservations at the specified data and time for the requested number of guests, find existing reservation allows for consumers who already have a reservation to search for their reservation information by inputting their email and the six digit code that they preset when creating their reservation, sign in allows administrators and managers to input their login credentials or allows managers to create a restaurant and an account for themselves.

The following is the specific implementations of each page of the application:

When a consumer requests to find an existing reservation, they are asked for their email and six-digit code that they preset when creating their reservation. Once they input that information, they are brought to another page where they can see their email, restaurant name that they booked the reservation at, the date and time that they booked, and the number of guests for the reservation. On this page there is also a button that they can click to allow them to cancel their reservation.

If a manager wants to create a restaurant, they click on the "sign in" button then the "create restaurant" link at the bottom of the page. They are then brought to a new page where they can input their desired username, password, restaurant name, and the street address. Once they click on the "create restaurant" button they are then given an alert stating if the restaurant was successfully created and they are sent back to the login page. If there was an error in creating the restaurant then they get an "error" alert, and the data is cleared from the page.

When you first go to the login page, there is a toggle switch which changes from admin login to manager login, and you set it to whichever account is being logged into. The username for admin is "admin" and the password is "password". Once the admin is logged in, they are taken to the "admin console" page. On this page they have two buttons, "log out" which logs them out and brings them back to the log in page, or "view all restaurants" which takes them to a new page; This page has a list of all the restaurants in the system with their name, address, open time, close time, and if they are active (1=active, 0=inactive). Under each restaurant info block there is a button to delete the restaurant or,

CS3733 Team Project Report

only if the restaurant is active, a button to show the availability report. Once you click on the “delete this restaurant” button you get an alert that states if the deletion was successful or not then the page is refreshed. When you click on the “show availability” you are taken to a new page. You are requested to input a date in which you want to view the data for. You then choose which table you wish to view the data for (you have to click the button twice). You can then select a time that already has a reservation and view all the data for that reservation at that time. If you select a time that does not have a reservation, you get an error message. There is also the option to cancel that specified reservation.

When a consumer clicks on “Browse Restaurant” button on the homepage, they are taken to a page that lists all the active restaurants. They can then click on the “Search Times” button underneath each restaurant to search open reservation times for that specified restaurant by date and time. It will then show all the tables at that restaurant and if they are reserved or not. You can then click on the “Make a reservation” button to select that reservation. You are then brought to a new page and must enter your email and all the information about the reservation that you want to reserve.

When a consumer clicks on “Search Restaurants”, they are brought to a page to input the date and time that they want to make a reservation for. When they press “submit” they are given all of the restaurants that have reservations at that time. You then click “make a reservation” under the restaurant. You are then brought to the same page as in the method described before to create the reservation. To find the table, you must click the “Search Times” button to find the table information. You are then brought back to the “Search Times” page so you can find the table information to submit on either this page or the previous “Search Restaurants” page. We did it this way simply because even though it is more time consuming for the consumer, our team ran out of time to make a smoother implementation.

When a manager logs-in to an inactivated or newly created restaurant they are brought to a page with the following functionality: they can view the information about the restaurant, change the opening and closing times of the restaurant, create a table with a specified table number and number of seats, view all the created tables for that restaurant, delete the restaurant, or activate the restaurant.

If the restaurant is active and the manager goes to log in, they are brought to an active homepage (it may bring you to the wrong page at first but will go to the proper page after a few seconds). On this page you will see the restaurant name, address, opening time, and closing time. You will also see the green “Active” sign to indicate that the restaurant is active. There is a button titled “Get Closed Dates” that, when pressed, if there are no closed dates nothing will show but if there are closed dates then those values will appear in a list formatting. There is a “Delete Restaurant” button in the bottom right corner of the page which will delete the restaurant and send you back to the log in page if successful. There is another input field titled “Close Future Date” in which you can input what future dates that you wish to close the restaurant. Once you submit the value you will get a success alert, and that date will show in the “closed dates” list. There will also be a “Reopen” button next to each of the closed dates which will re-open the restaurant on that date and delete that value from the list then refresh the page. It is important to note that occasionally the

CS3733 Team Project Report

formatting of this page gets altered in which the “Active” sign will be moved to the left side of the page, when this happens there is no way to fix it but all the functionalities will still work.

Some pages will have a working “back” button which goes back one page, but there are no buttons that go back to the opening homepage. To do so, you can either press the built-in browser back-button until you reach that page or manually change the URL so that it goes back to the original page.

Team organization, members, and responsibilities

We held in-person meetings at least 4-5 times a week for 2+ hours at a time. These were working sessions where we discussed any problems we were facing on our own. Once we worked through the problems, we worked on our use cases and discussed issues or how we were implementing the use cases, so each member understood how everything was connected. We did not have specified sub-teams, but each member played to their strengths to get work done as quickly and efficiently as possible.

Each person held the following roles in and created the specified use cases:

- Jacob Bergeron: Assisted in creating the mySQL database, created the AWS accounts
- Skyler Dooley: Assisted in creating the mySQL database, wrote the reports, worked mainly on consumer browse restaurants and consumer find existing reservation
- Michael Monda: Managed GitHub repository, worked mainly on availability report and admin view restaurants
- Jacob Newman: Did most of the initial css and HTML work, worked on availability report

The front and back-end of each use cases was done in entirety by 1-3 people. Most of the time, each person who is listed next to each use case created the lambada functions, API Gateway, front-end and back-end. However, sometimes others would go back into other functions to fix small HTML or functionality issues.

The following was written using our “To Do” List:

Manager Use Cases:

- CreateRestaurant done by Jacob B
- LoginManager done by Jacob N
- EditTime done by Jacob N
- ActivateRestaurant done by Jacob N and Jacob B
- EditTableInfo done by Skyler and Jacob N
- ShowTableInfo done by Jacob N
- CreateTable done by Skyler and Jacob N
- DeleteTable done by Jacob N
- ActivateRestaurant done by Jacob B
- DeleteRestaurant done by Skyler
- ShowClosedDates done by Skyler and Jacob N

CS3733 Team Project Report

- CloseDay done by Jacob B and Michael
 - Popup done by Jacob N
- ReopenDay done by Jacob B, Jacob N, and Michael
 - Popup done by Jacob N
- Review Availability done by Michael and Jacob N

Admin Use Cases:

- AdminHomepage done by Michael
- LoginAdmin done by Jacob N
- AdminListRestaurants done by Michael
- DeleteRestaurants done by Michael
- Generate Availability Report done by Michael and Jacob N
- Cancel Reservations done by Michael and Jacob N

Consumer Use Cases:

- ConsumerListRestaurants done by Skyler
- Search Specific Restaurant done by Jacob B
- Search Times done by Jacob B
- MakeReservation done by Jacob B
- FindExistingReservation done by Skyler
- CancelExistingReservation done by Skyler and Jacob B

Process

At the start, we met to get some ‘boiler plate’ code done that we could build upon. We did this by meeting in person and using VSCode’s live share feature to all work on the project at the same time. We did this via live share because we felt it was easiest to make rapid changes that we could all see at the start. Eventually we did move to having each individual person working on their own branches in the Git Hub repository and each person would create a pull request when they wanted to merge their branch with the main branch. If there was more than one person working on the same functionality for an extended period of time, we would go back to using live share so that it was easier to work together and get that implementation working quicker.

After this we split up responsibilities based roughly on use cases and worked individually to try and implement these use cases. We would meet 5 times a week to talk about where we were at and try and debug any problems that we may have been facing.

We set deadlines for ourselves to get the use cases done by a certain time so that we could start working on the use cases that used those pages that were being created.

We communicated primarily through iMessaging via our phones. Everyone was very responsive, and we were able to resolve issues relatively quickly when they arose.

Tools

Microsoft Word, VSCode, VSCode live share, GitHub, GitHub co-pilot, AWS, MySQL

CS3733 Team Project Report

- We attempted to use AWS Amplify to host our code but that did not work for us, instead we had to change how we passed data between pages in order for it to work on AWS. We ended up using the AWS S3 bucket instead to host the app.
- We used GitHub as a communal workspace for the group members so that our code was all saved in the same area. This worked very well for our team.
- We used GitHub co-pilot to help us figure out how to write some areas of the code where we were unsure how to approach. This helped smooth the coding process and make things go faster other than waiting for a response from other students or the professor.
- We used VSCode live share when multiple people had to work on the same code implementation which allowed us to seamlessly work together on the code.
- MySQL was quite easy for everyone in the group to use to ensure that they had all necessary information to write the lambda functions.
- We used Microsoft Word to keep track of our reports and “To Do” list which hosted all the required deliverables and use cases and who did what implementations.

Accomplishments

Overall, we completed the use cases that were stated previously in this document how we intended to from the beginning, with slight alterations through to simplify the implementation for when we needed to get the project finished on time. We were not able to accomplish anything outside the requirements due to time restraint.

We were not able to properly finish the generate availability report aspect of the project due to time constraints. This page shows the time, seats at the table, the number of seats filled, and the number of available seats at the table at that time. You can also cancel the specified reservation at that time.

Deliverables

Our deliverables are the following: README file, zip file containing all back-end lambda functions, text file consisting of the Git history, zip file containing all the front-end React /src files, and a URL for the entry points to the application hosted on AWS.

Reflection

What worked, what didn't work

Splitting up the work into use cases worked well for our team as it allowed us to work on the project on our own times and not mess up the progress of another teammate. We found that the only time that we got the most done was when we were all sitting down working together, but when it came to small and tedious tasks, we were able to branch off and work on those ourselves since they did not require input from other aspects of the program.

Our biggest mistake

Our biggest mistake was not starting the project earlier which caused us to not meet the requirements for iteration 1. We also did not implement our app on AWS until the night

CS3733 Team Project Report

before iteration 1 was due, which caused us to not realize how we were sending data between pages did not work with AWS. To avoid these issues, we should have started working on the coding aspect of the project at least 2 days prior and implement the application on AWS sooner to identify and solve the prevalent issues. We should have also started the availability report sooner because it proved to be one of, if not, the most complicated use case that we had to tackle.

Changes we would make

Start simple from the beginning to get it started quicker.

More videos/class time spent on AWS implementation and some suggestions on how to implement certain aspects of the project. We spent more time learning how to use AWS and debugging simple aspects of our code that could have been avoided if we understood what we were doing and what implementations work and do not work with AWS.

Lessons learned

Make sure that everyone effectively communicates what they are working on so that it does not negatively impact another person's work. Also communicate changes that are made so that everyone is aware of those changes and how they may affect something that was already written.

Things we learned

It was crucial for our team to work closely to make sure all the implementations work with one another so that we do not run into as many issues later in the process. This also ensured that everyone agreed on the implementations and how the entire process would go, which allowed everyone to be able to fully understand what was going on and play to their best strengths.

Advice to future teams

Get started EARLY. If we had started just two days earlier, we would have had more time to get iteration 1 completed by the deadline and wouldn't have had to rush to get it implemented on AWS last minute and then find out we had problems we could not solve on our own in one night.

Also, implement the project on AWS early to identify issues early on. One thing that I found helped our group work was meeting every weekday for AT LEAST two hours (one hour outside of class) and sitting together to get use cases done and quickly ask and get questions answered from groupmates.

We also were able to split some implementation up by having two people work on one difficult use case. It was extremely helpful that every member did both front and back-end implementation of the use cases that they worked on, this allowed all of us to understand all parts of the project and could have multiple heads work on one issue if necessary. As the end of the term got closer, we would split off at the end of the day and work on use cases that we would claim so that everyone would have something to work on.

CS3733 Team Project Report

We also created a “To do” document which contained all the use cases that we had to do and wrote who would work on it next to the title. We would then highlight the use cases if they were finished, someone was working on it, or if they were not working and the person who was originally working on it needed another pair of eyes. In the “To do” document we would specify the use case by user and what page it was supposed to go on which greatly helped us stay organized. We would always try to do all our work together during the day and then at night if we had left-over problems from the day, we would try to solve them on our own then come back the next day to talk about if we solved the issue or what was not working. We would also keep each other updated via text on what we were currently working on and questions or comments that we had. All our data was uploaded via GitHub, and it helped to have one person who understood how to get that up from the beginning.